JS Fundamentos do JavaScript I – Introdução, Variáveis e Operadores

Nesta aula, exploraremos os conceitos básicos de JavaScript, abordando tipos primitivos, variáveis e operadores – desde os aritméticos até os lógicos e de comparação. Além disso, veremos exemplos práticos para demonstrar como aplicar cada um desses recursos na linguagem.

"Com grandes poderes vem grandes responsabilidades." - Tio Ben (Stan Lee/Marvel Comics)

1. História e Evolução do JavaScript

1995
Criado por **Brendan Eich** na Netscape em apenas 10 dias.

Inicialmente chamado de **Mocha**, depois **LiveScript** e finalmente **JavaScript**.

1997
JavaScript é padronizado pela ECMA International como **ECMAScript**.

2009
ES5 introduz importantes adições como JSON.parse e strict mode.

2015
ES6/ES2015 traz let/const, arrow functions, classes, promises e muito mais.

2016-2024
Lançamentos anuais com novas funcionalidades.

JavaScript se expande para servidores (Node.js), aplicações móveis e desktop.

JavaScript evoluiu de uma linguagem simples para uma plataforma completa de desenvolvimento:

Navegadores

De pequenos scripts a aplicações web completas (SPAs).

Servidores

Node.js permitiu JavaScript no backend.

Mobile

React Native, Ionic.

Desktop

Electron.

IoT

JavaScript em diversos dispositivos.

JavaScript no Navegador

Motores JavaScript

Os navegadores modernos executam JavaScript através de motores altamente otimizados:

- **V8**: Desenvolvido pelo Google, usado no Chrome, Edge e Node.js
- **SpiderMonkey**: Usado no Firefox
- **JavaScriptCore**: Usado no Safari

Características

Características importantes incluem:

- APIs do navegador: DOM, Window, Fetch, localStorage
- Runtime único (single-threaded) com event loop
- Interação com HTML e CSS para manipulação da página

3. Formas de Incluir JavaScript no HTML

Existem diversas maneiras de inserir JavaScript em um documento HTML:

Script interno

```
<script>
  console.log("Script interno");
</script>
```

Script externo

```
<script src="script.js"></script>
```

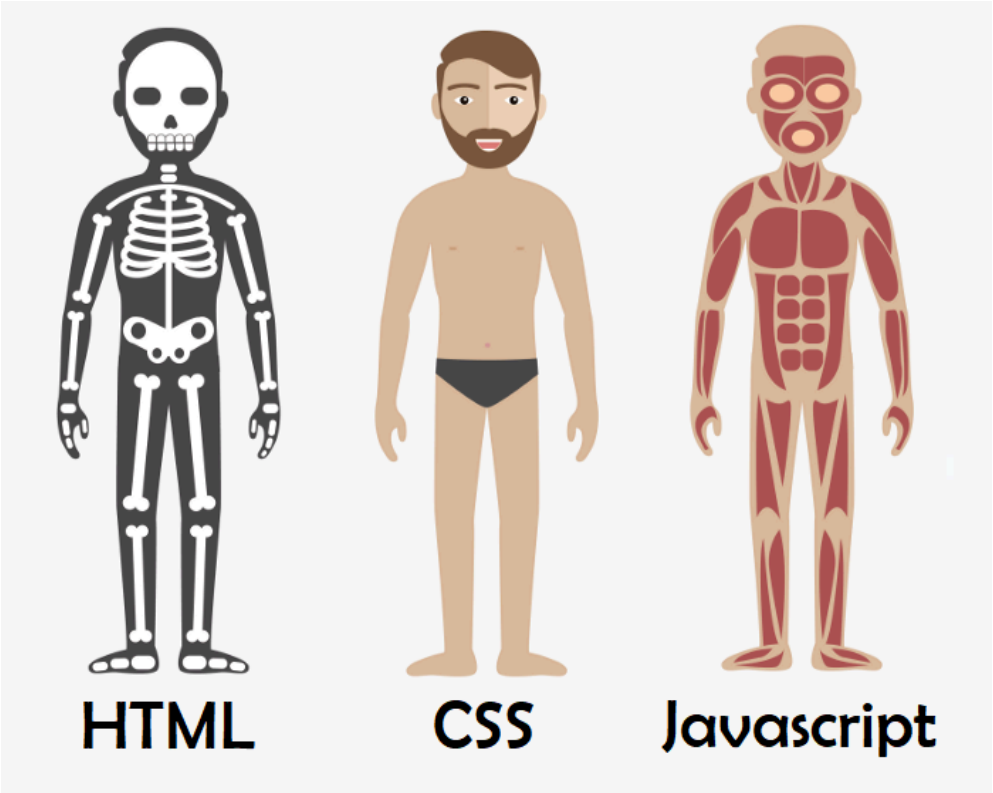
Com atributo defer

```
<script src="script.js" defer></script>
```

Com atributo async

```
<script src="script.js" async></script>
```

🚀 4. Primeiros Passos com JavaScript



💓 5. HTML, CSS e JavaScript: A Analogia do Corpo Humano

Pense nas tecnologias web como partes de um corpo humano:

👤 HTML (Esqueleto)	🖌️ CSS (Pele e Aparência)

Define a estrutura básica da página, assim como os ossos definem o corpo.

Responsável pela aparência visual, controlando cores, fontes e layout.

JavaScript (Músculos)

Adiciona interatividade e dinamismo, permitindo que os elementos respondam a eventos.

6. Console para Debugging

O console do navegador (F12 em muitos navegadores) é fundamental para registro de mensagens e depuração:

Exemplos de uso do console

```
// Métodos básicos
console.log("Mensagem normal");
console.info("Informação");
console.warn("Aviso");
console.error("Erro");

// Tabelas
console.table([
  { id: 1, tarefa: "Estudar JS" },
  { id: 2, tarefa: "Praticar código" }
]);

// Agrupamento
console.group("Grupo de logs");
console.log("Log 1");
console.log("Log 2");
console.groupEnd();
```

```
// Medição de tempo
console.time("Timer");
// ...algum processamento...
console.timeEnd("Timer");
```

7. Comentários e Case Sensitivity

Comentários

```
// Comentário de linha única

/* Comentário
   de múltiplas linhas */
```

JavaScript diferencia maiúsculas de minúsculas, portanto `nome` e `Nome` são variáveis distintas.

8. Variáveis em JavaScript

Existem três principais formas de declarar variáveis em JavaScript: `var`, `let` e `const`. Nas versões modernas de JavaScript, `let` e `const` são preferíveis.

Exemplos de Declaração

```
// Com var (escopo de função)
var antigo = "Forma antiga de declarar variáveis";

// Com let (escopo de bloco, pode ser reatribuída)
let userName = "Maria";
userName = "João"; // permitido

// Com const (escopo de bloco, não pode ser reatribuída)
const API_URL = "https://api.example.com";
// API_URL = "..."; // Erro
```

Escopo: `var` ignora o escopo de bloco (exceto em funções), enquanto `let` e `const` {" "} respeitam o escopo de bloco.

9. Tipos de Dados Primitivos

JavaScript possui os seguintes tipos de dados primitivos:

- `string` (textos)
- `number` (números)
- `boolean` (verdadeiro ou falso)
- `undefined` (valor não atribuído)
- `null` (valor nulo intencional)
- `symbol` (identificador único - ES6)
- `bigint` (inteiros muito grandes - ES2020)

Exemplos de Declaração

```
let texto = "Olá";
// String: sequência de caracteres.

let numero = 42;
// Number: valores numéricos.

let isCompleted = false;
// Boolean: verdadeiro ou falso.

let semValor;
// Undefined: variável declarada sem valor.

let nulo = null;
// Null: ausência intencional de valor.
// (typeof null retorna "object", comportamento histori

let uniqueId = Symbol("id");
// Symbol: cria um identificador único.

let bigNumero = 999999999999999999999999n;
// BigInt: para inteiros muito grandes.
```

Verificação de Tipos com `typeof`

Cada `console.log` exibe o valor da variável seguido do seu tipo:

- **texto**:
  `console.log(texto, typeof texto);`
  Saída: "Olá" "string" — 'texto' é uma string.
- **numero**:
  `console.log(numero, typeof numero);`
  Saída: 42 "number" — 'numero' é um número.
- **isCompleted**:
  `console.log(isCompleted, typeof isCompleted);`
  Saída: false "boolean" — 'isCompleted' é um booleano.
- **semValor**:
  `console.log(semValor, typeof semValor);`
  Saída: undefined "undefined" — 'semValor' não foi atribuído.

- **nulo**:
  `console.log(nulo, typeof nulo);`
  Saída: null "object" — 'nulo' é null, mas o `typeof` retorna "object".
- **uniqueId**:
  `console.log(uniqueId, typeof uniqueId);`
  Saída: Symbol(id) "symbol" — 'uniqueId' é um símbolo único.
- **bigNumero**:
  `console.log(bigNumero, typeof bigNumero);`
  Saída: 999999999999999999999999n "bigint" — 'bigNumero' é um BigInt.

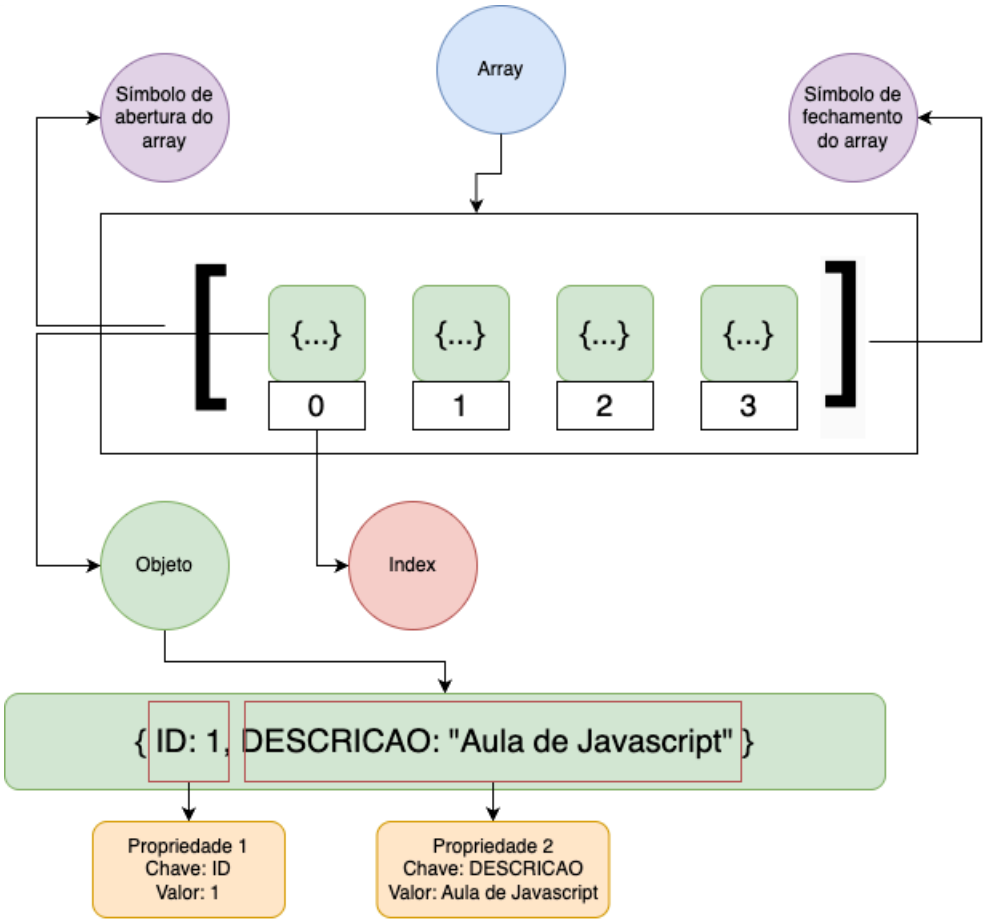
Essas declarações e verificações são úteis para compreender como o JavaScript interpreta os diferentes tipos de dados. Note, por exemplo, que embora `null` represente a ausência de valor, o operador `typeof` retorna "object", um comportamento histórico da linguagem.

10. Tipos de Dados Complexos

Nesta seção, vamos explorar como os dados complexos são estruturados e utilizados em JavaScript. Através de exemplos práticos e de um diagrama ilustrativo, você entenderá como objetos, arrays, datas e expressões regulares podem ser combinados para resolver problemas reais e organizar informações de forma eficiente.

Visualizando Estruturas: Array de Objetos

Imagine que você tem uma caixa capaz de armazenar diversos itens, onde cada item tem suas próprias características. Em JavaScript, essa caixa é um **array**, e cada item é um **objeto** com suas propriedades e métodos. O diagrama abaixo ilustra essa organização:



Array: Delimitado por `[` e `]`, indexando objetos a partir do índice 0.

Objeto: Estruturado entre `{` e `}`, contendo pares chave-valor (por exemplo, `id: 1`).

Esta visualização reforça como cada objeto dentro do array mantém sua estrutura interna, contribuindo para uma organização clara e intuitiva dos dados.

Exemplos Práticos de Dados Complexos

Objetos (Object)

Os objetos são coleções de pares chave-valor que representam entidades com atributos e comportamentos. No exemplo abaixo, um objeto `task` define uma tarefa com propriedades e um método para alternar seu estado:

```
let task = {
  id: 1,
  title: "Aprender JavaScript",
  completed: false,
  assignee: {
    name: "João",
    email: "joao@example.com"
  },
  toggleComplete() {
    this.completed = !this.completed;
  }
};
```

Vetores (Arrays)

Arrays são listas que armazenam múltiplos itens, como várias tarefas. Cada elemento do array pode ser um objeto, permitindo a manipulação organizada de conjuntos de dados:

```
let tasks = [
  { id: 1, title: "Tarefa 1", completed: false },
  { id: 2, title: "Tarefa 2", completed: true }
];
```

Datas (Date)

O objeto `Date` facilita a manipulação de datas e horários, permitindo criar instâncias que representam o momento atual ou datas futuras para agendamentos e eventos:

```
let hoje = new Date();
let dataFutura = new Date("03-26-2025");
```

Expressões Regulares (RegExp)

Expressões regulares são utilizadas para validar e extrair padrões em strings. No exemplo a seguir, uma expressão regular verifica se um número de telefone brasileiro está no formato correto, seja com prefixo de 4 ou 5 dígitos:

```
let pattern = /^\+55\(\d{2}\)\d{4,5}\-\d{4}$/;
console.log(pattern.test("+55(11)92299-0899")); // true
console.log(pattern.test("+55(11)2299-0899"));  // true
```

11. Conversão de Tipos

É possível converter dados manual ou implicitamente:

Para String

```
String(42); // "42"
42 + "";  // "42"
```

Para Number

```
Number("42"); // 42
parseInt("42.9");  // 42
parseFloat("42.9"); // 42.9
```

Para Boolean

```
Boolean("texto"); // true
Boolean("");       // false
```

Falsy: `""` , `0` , `null` , `undefined` , `NaN` e `false` são considerados valores false ao converter para boolean.

12. Operadores em JavaScript

Operadores Aritméticos

(+, -, *, /, %, **)

```
let a = 10;
let b = 3;
console.log(a + b); // 13
console.log(a - b); // 7
console.log(a * b); // 30
console.log(a / b); // 3.333...
console.log(a % b); // 1
console.log(a ** b); // 1000
```

Esses operadores são usados para realizar cálculos matemáticos básicos. O símbolo `%` (módulo) retorna o resto da divisão de dois números, e `**` representa exponenciação.

Operadores de Incremento/Decremento

(++, --)

```
let contador = 0;
console.log(++contador); // 1 (pré-incremento)
console.log(contador++); // 1 (pós-incremento, agora cor
```

Pré-incremento (`++x`): incrementa antes de retornar o valor. Pós-incremento (`x++`): retorna o valor antigo e depois incrementa. O mesmo vale para decremento (`--`).

Operadores de Atribuição

(=, +=, -=, *=, /=, %=, **=)

```
let x = 10;
x += 5; // x = 15
x -= 3; // x = 12
x *= 2; // x = 24
x /= 4; // x = 6
x %= 2; // x = 0
x **= 3; // x = 0 (0 elevado a 3)
```

Esses operadores simplificam a forma como atribuímos valores a variáveis, combinando atribuição com a operação.

Operadores de Comparação

(==, ===, !=, !==, >, <, >=, <=)

```
console.log(10 == "10");   // true (compara valor soment
console.log(10 === "10");  // false (compara valor e tip
console.log(5 != "5");     // false (valor é igual)
console.log(5 !== "5");    // true (valor igual, mas tip
```

A diferença fundamental entre `==` e `===` (e entre `!=` e `!==`) é que a versão com **três** sinais de igual (ou exclamação) verifica tanto o valor quanto o tipo, enquanto as versões com **dois**{" "} sinais realizam coerção de tipo antes de comparar.

Operadores Lógicos

(&&, ||, !)

```
console.log(true && false);  // false (AND)
console.log(true || false);  // true  (OR)
console.log(!true);          // false (NOT)
```

- `&&` (AND): retorna true apenas se ambas as condições forem true.
- `||` (OR): retorna true se pelo menos uma das condições for true.
- `!` (NOT): inverte o valor booleano.

Operador Ternário

```
let idade = 18;
let status = idade >= 18 ? "Maior de idade" : "Menor de
```

O operador ternário permite expressar condicionais simples em uma linha, retornando um valor caso a condição seja verdadeira e outro caso seja falsa.

Coalescência Nula (??)

```
let valor = null;
let resultado = valor ?? "Valor padrão"; // "Valor padrá
```

Retorna o operando da direita apenas se o da esquerda for{" "} `null` ou `undefined`. É diferente de `||`, que trataria qualquer valor "falsy" (como 0 ou "") como falso.

Encadeamento Opcional (?.)

```
let usuario = { nome: "Ana" };
console.log(usuario?.endereco?.rua); // undefined
```

Permite acessar propriedades aninhadas sem causar erros se algum nível for `null` ou `undefined`.

Operador Spread (...)

```
let arr1 = [1,2];
let arr2 = [3,4];
let combinado = [...arr1, ...arr2]; // [1,2,3,4]
```

Expande iteráveis (arrays, objetos) em locais onde múltiplos elementos são esperados, útil para **clonar** ou **mesclar** dados.

13. Exercícios Práticos

Exercício 1: Variáveis Pessoais

Crie duas variáveis (nome e idade). Exiba a frase: "Meu nome é [nome] e tenho [idade] anos." no console.

▶ **Ver solução**

Exercício 2: Operações com Variáveis

Crie duas variáveis numéricas (`valor1` e `valor2`) e exiba soma, subtração, multiplicação e divisão.

▶ **Ver solução**

Exercício 3: Representando uma Tarefa

Crie variáveis para representar uma tarefa (título, descrição e status booleano). Mostre essas informações no console.

▶ **Ver solução**

Exercício 4: Debugging no Console

Copie o trecho abaixo e observe o resultado no console. Por que o resultado é "105" ao invés de 15?

```
let numero1 = 10;
let numero2 = "5";
console.log(numero1 + numero2);
```

Explique a razão no comentário do próprio código. Em seguida, modifique-o para realizar a soma corretamente.

▶ **Ver solução**

Exercício 5: Template String

Crie uma variável `versaoApp` (número) e exiba a mensagem: "Bem-vindo ao aplicativo [nome] versão [versaoApp]" usando **template strings**.

▶ **Ver solução**

Exercício 6: Operadores de Comparação

Peça ao usuário dois valores (pode usar `prompt()` ou atribuições diretas) e compare usando `==`, `===` e `>`. Mostre no console o resultado de cada comparação.

▶ **Ver solução**

Exercício 7: Incremento de Prioridade

Crie uma variável `prioridade` (valor inicial 3) e use operadores de incremento e atribuição para aumentar até 5. Exiba no console cada passo.

▶ **Ver solução**

14. Conclusão

Nesta aula unificada, você aprendeu:

- Como o JavaScript surgiu e evoluiu
- Como ele se integra ao HTML/CSS e ao navegador
- Como declarar variáveis (`var` , `let` , `const`)
- Tipos de dados primitivos e complexos
- Formas de conversão de tipos
- Uso do console para debugging
- Principais operadores (aritméticos, lógicos, etc.)

Com esses fundamentos, você estará pronto para aprofundar ainda mais no desenvolvimento de aplicações ricas e interativas em JavaScript.